

Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration

Task 1: AI-Assisted Pair Programming

Prompt:

1. Create a simple Python program to track daily expenses.
2. Each expense should have amount and category.
3. Store multiple expenses and display them.
4. Add a feature to calculate monthly total expenses.
5. Improve code with validation and functions.

Code:

A screenshot of a code editor window titled 'lab21.py > ...'. The editor contains a Python script for tracking expenses. The script defines a list 'expenses' and three functions: 'add_expense()', 'show_expenses()', and 'total_expense()'. It uses a 'while True' loop to handle user input for adding, showing, or calculating total expenses. The code is as follows:

```
1 expenses = []
2
3 def add_expense():
4     amount = float(input("Enter amount: "))
5     category = input("Enter category: ")
6     expenses.append((amount, category))
7
8 def show_expenses():
9     for exp in expenses:
10        print("Amount:", exp[0], "Category:", exp[1])
11
12 def total_expense():
13     total = 0
14     for exp in expenses:
15         total += exp[0]
16     print("Total Expense:", total)
17
18 while True:
19     print("1.Add 2.Show 3.Total 4.Exit")
20     ch = int(input("Enter choice: "))
21
22     if ch == 1:
23         add_expense()
24     elif ch == 2:
```

Output:

```
PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:\Users\rahul\Desktop\AI Assistanting coding\venv\Scripts\Activate.ps1"
(.venv) PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:\Users\rahul\Desktop\AI Assistanting coding\venv\Scripts\python.exe" "c:/Users/rahul/Desktop/AI Assistanting coding/lab21.py"
1.Add 2.Show 3.Total 4.Exit
Enter choice: 1
Enter amount: 500
Enter category: food
1.Add 2.Show 3.Total 4.Exit
Enter choice: 2
Amount: 500.0 Category: food
1.Add 2.Show 3.Total 4.Exit
Enter choice: 200
(.venv) PS C:\Users\rahul\Desktop\AI Assistanting coding> & "c:\Users\rahul\Desktop\AI Assistanting coding\venv\Scripts\python.exe" "c:/Users/rahul/Desktop/AI Assistanting coding/lab21.py"
1.Add 2.Show 3.Total 4.Exit
Enter choice: 1
```

Explanation:

1. This program stores expenses using a list.
2. User can add, view, and calculate total expenses.
3. Functions are used to organize the code properly.

Task 2: Version Control Integration (Git)

Prompt:

1. Create a student grade calculator program.
2. Add marks for different subjects.
3. Calculate average and percentage.
4. Add grade classification like A, B, C.
5. Use Git to manage changes with branches.

Code:

```
1 marks = []
2
3 n = int(input("Enter number of subjects: "))
4
5 for i in range(n):
6     m = int(input("Enter marks: "))
7     marks.append(m)
8
9 total = sum(marks)
10 avg = total / n
11
12 print("Total:", total)
13 print("Average:", avg)
14
15 if avg >= 90:
16     print("Grade: A")
17 elif avg >= 75:
18     print("Grade: B")
19 elif avg >= 50:
20     print("Grade: C")
21 else:
22     print("Grade: Fail")
```

Output:

```
>>> %Run -c $EDITOR_CONTENT
Enter number of subjects: 3
Enter marks: 90|
Enter marks: 80
Enter marks: 70
Total: 240
Average: 80.0
Grade: B
```

Explanation:

1. This program takes marks as input from user.
2. It calculates total and average marks.
3. Based on average, grade is assigned.

Task 3 – AI for Commit Message Generation

Prompt :

1. Create a password validation program.
2. Check minimum length of password.
3. Ensure at least one special character exists.
4. Reject weak passwords.
5. Generate commit messages for changes.

Code:

```
digits2.py x <untitled> * x
1 password = input("Enter password: ")
2
3 if len(password) < 6:
4     print("Weak password")
5 elif not any(char in "!@# $" for char in password):
6     print("Add special character")
7 else:
8     print("Strong password")
```

Output:

```
Shell ×  
  
>>> %Run -c $EDITOR_CONTENT  
Enter password: abc123  
Add special character  
>>>
```

Explanation:

1. This program checks password strength.
2. It verifies length and special characters.
3. Weak passwords are rejected.

Task 4 – Pair Programming Simulation

Prompt :

1. Create a quiz program with questions and answers.
2. Take answers from user.
3. Calculate score based on correct answers.
4. Add input validation for answers.
5. Use Git for collaboration and merging.

Code:

```
questions = ["Capital of India?", "2+2=?"]  
answers = ["Delhi", "4"]  
  
score = 0  
  
for i in range(len(questions)):  
    user = input(questions[i] + " ")  
  
    if user.lower() == answers[i].lower():  
        score += 1  
  
print("Your Score:", score)
```

Output:

```
>>> %Run -c $EDITOR_CONTENT
Capital of India? delhi
2+2=? 4
Your Score: 2
>>>
```

Explanation:

1. This program asks questions to user.
2. It checks answers and gives score.
3. Input is compared using simple logic.